

The background image shows a large industrial warehouse with high ceilings and metal shelving units filled with yellow boxes. In the foreground, there are several yellow robotic arms (likely KUKA) and a yellow mobile robot (AGV) with a robotic arm. The scene is dimly lit, with the primary light source coming from the overhead industrial lights, creating a warm, yellowish glow. The text is overlaid on this image.

Multi-Agent Cooperation to Accomplish a Sequence of Tasks using Deep Reinforcement Learning

Undergrad Researcher: Sreeyutha Ratala

Mentors: Nabil Khan, Prof. Maria Gini, Prof. Chris Umans

Fellowship: Caltech SURF

Contents

- ▣ Intro to Reinforcement Learning and Multi-Agent Systems
- ▣ Problem Addressed and Methods Used
 - Deep Q-learning and Multi-layered Neural Network
- ▣ Experimental Results
- ▣ Conclusions and Future Work

1. Introduction

Motivation

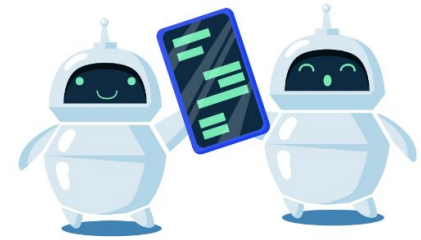
- ▣ Real world problems mandate teamwork to solve **complex tasks**
 - Organizations, sports, warehouses.
 - Ex: monitoring large areas, moving bulky objects, backup workers.
- ▣ Automating complex tasks requires **multi-agent cooperation** and decomposition.
 - Robots cooperate and work together.



What is a Multi-Agent System (MAS)?

Agent : a program, with senses, that is capable of making decisions.

MAS : a system with multiple agents interacting in an environment to achieve a common goal.



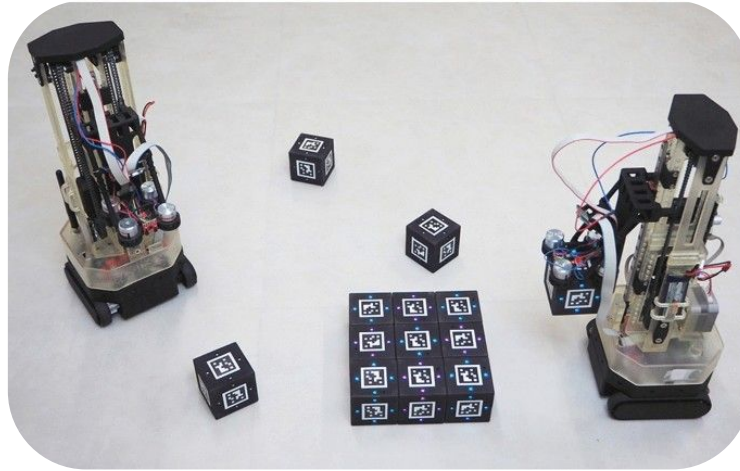
Advantages

- ▣ Increased **efficiency, diversity, accuracy, & scalability**
 - Generate more data, sharing knowledge
- ▣ Handle **complex tasks**, overcoming physical limitations
- ▣ Increased **flexibility**
 - Tolerate failure of individual agents

Disadvantages

- ▣ Designing & maintaining MAS very complex
 - Conflicting goals and **complex coordination**
- ▣ **Increased uncertainty** in changing environment
 - Handle incomplete information

Research Question



Will agents tasked with a novel problem learn to cooperate?

If so, does it scale well to different numbers of robots and varying environments? What is the learning curve?

Specific Problem

Can robots learn to accomplish a sequence of complex tasks without being given specific instructions?

- *Warehousing Problem*: find and move objects to a goal location.
- Simulated Environments
 - Varying grid sizes
 - Varying initial positions of agents, artifacts, and goal
- Different numbers of robots and artifacts to move.
 - How will the system scale?
- Extends Nabil Khan's master's thesis.

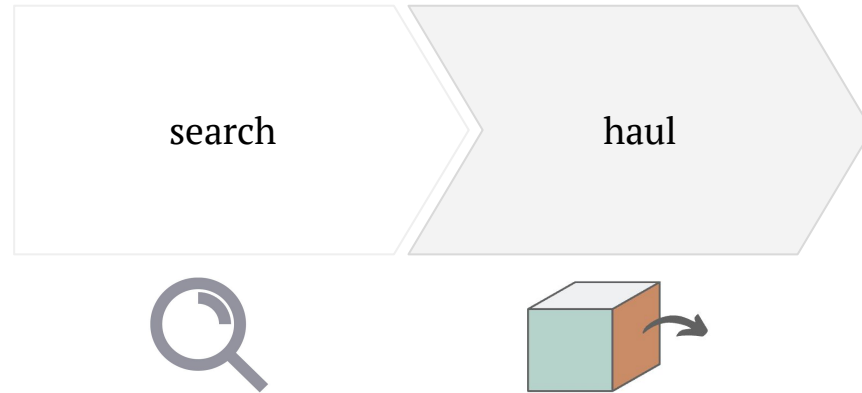


2. My Work

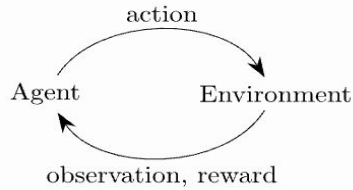
*Can agents learn to cooperate without pre-allocating
tasks or using communication?*

Setup

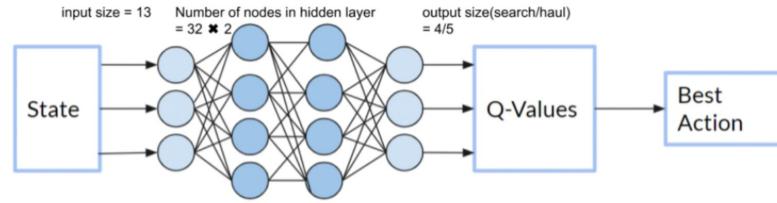
Task: agents must learn to **find** bulky artifacts and **move** them to their destination, which requires cooperation in pairs.



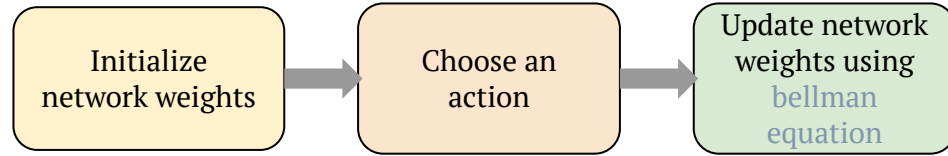
Deep Reinforcement Learning with Deep Q-Learning and Multi-Layered Neural Network



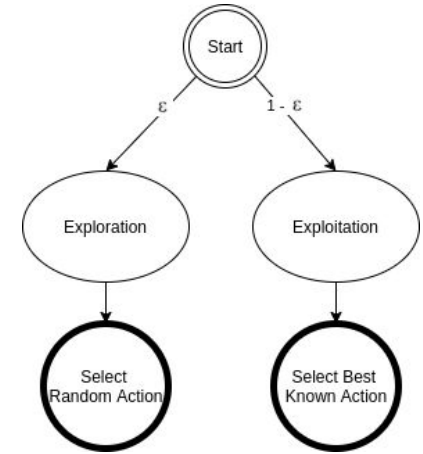
Classic Agent-Environment Loop for RL Algorithm Implementation



Neural network Architecture



DQN Algorithm



Epsilon-Greedy Exploration Strategy

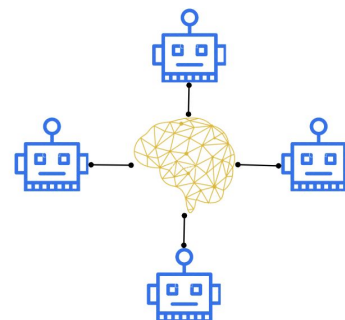
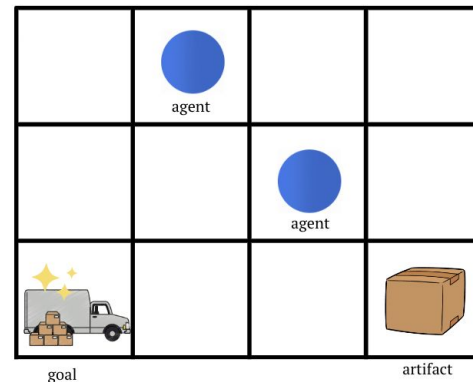
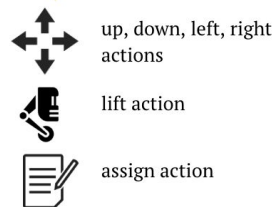
$$\text{New } Q(s, a) = Q(s, a) + \alpha [R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Labels for the equation components:

- $\text{New } Q(s, a)$: New Q value for the state and action
- $Q(s, a)$: Current Q values
- α : Learning Rate
- $R(s, a)$: Reward for taking an action in a state
- γ : Discount Rate
- $\max_{a'} Q(s', a')$: Maximum expected future reward
- $Q(s, a)$: Current Q values

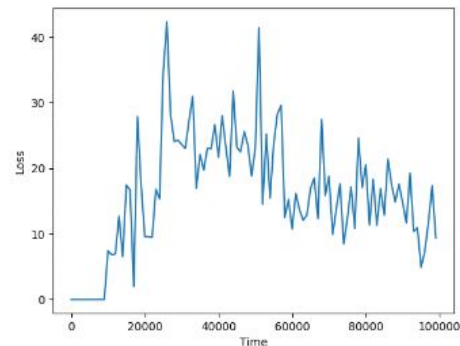
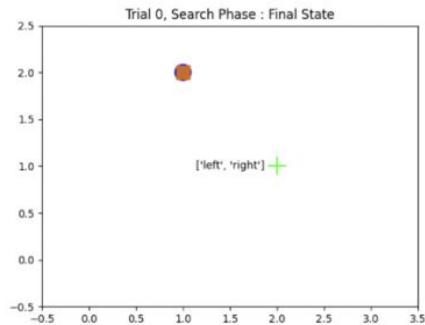
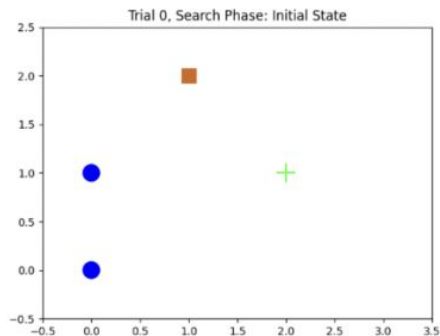
Environment & Architecture

- $n \times m$ grid, discrete space with (x, y) coordinates
- Robots r_1 and $r_2 \in R$ randomly placed
- Artifacts l_i, l_{i+1}, l_{i+k} randomly placed
 - Each artifact is a sub-task and is homogeneous.
- Fixed goal location g .
- Action space A : {"up", "down", "left", "right", "lift", "assign"}
- Robots must learn sequence of actions in A to find and move artifacts to g .
 - Synchronous movement required to move an artifact.
- **Centralized training**: Agents share neural network and replay memory
- **Distributed Control**: Agents execute tasks independently and synchronized.

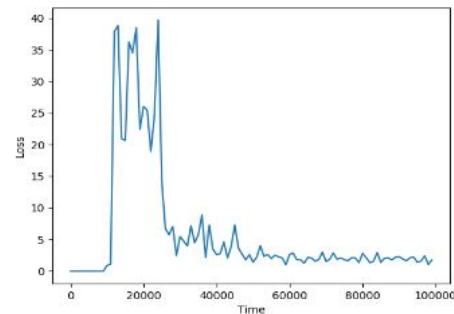
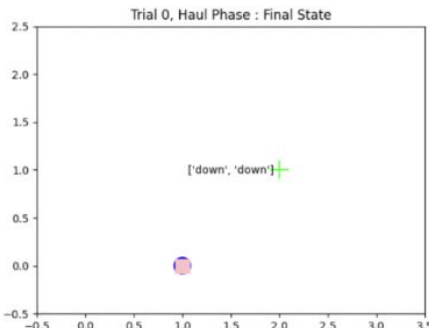
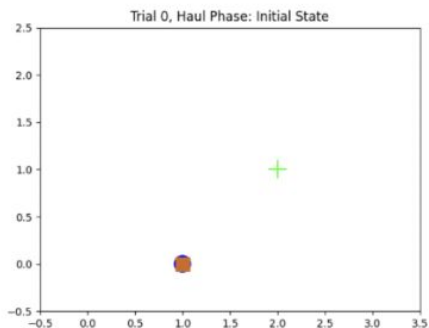


3. Experimental Results

Preallocation: Tasks allocated prior to training



Search Task Training: loss vs time



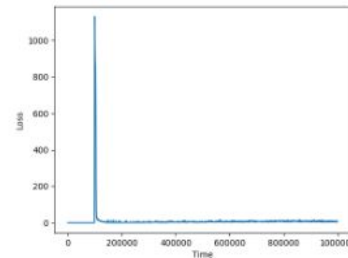
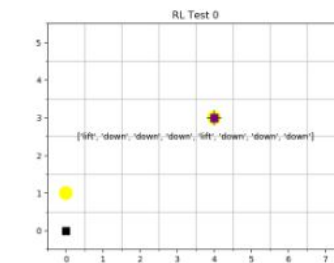
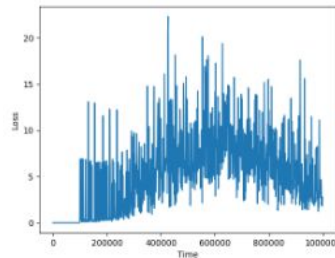
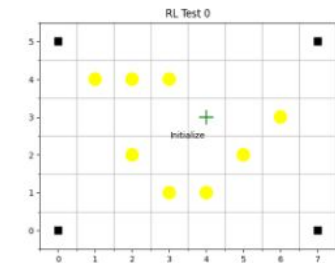
Haul Task Training: loss vs time

Initial States

Final States

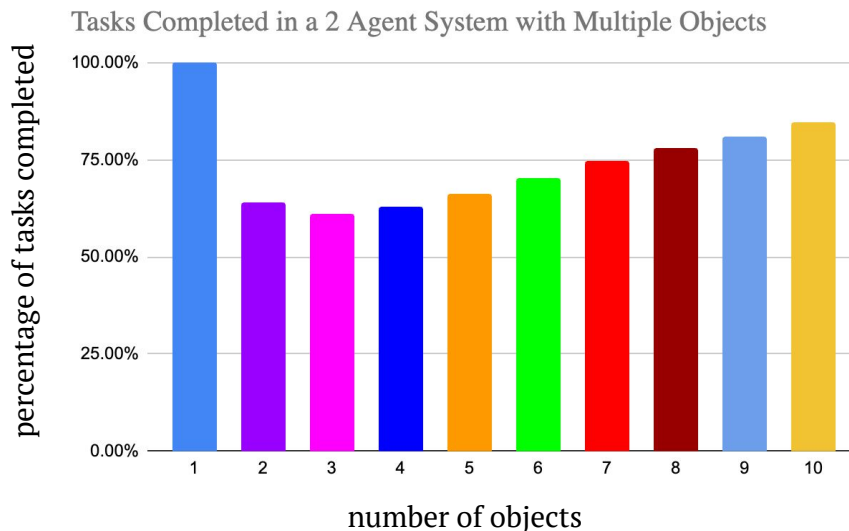
Multiple Agents Moving Objects, Tasks allocated prior to training

- ▣ Parallel Task Completion
- ▣ Result: Success
 - Robots learned cooperative behavior to move artifacts to destination
 - Scalability limited in terms of the number of robots and tasks, and training time efficiency.
 - Q-learning algorithm maximized the rewards



Moving Multiple Objects, Tasks allocated prior to training

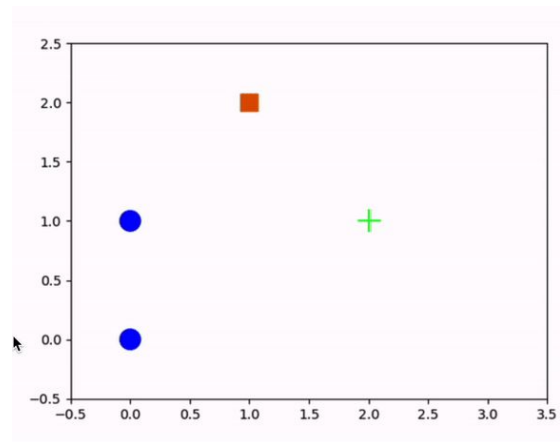
- ▣ Model trained with 1 artifact used to test performance of moving multiple artifacts
 - Artifacts are **homogeneous**
 - Cannot scale up number of agents from single training as parallel actions must be learned prior to the training.
- ▣ Result: Success
 - Tasks completed most of the time.



Extending the System

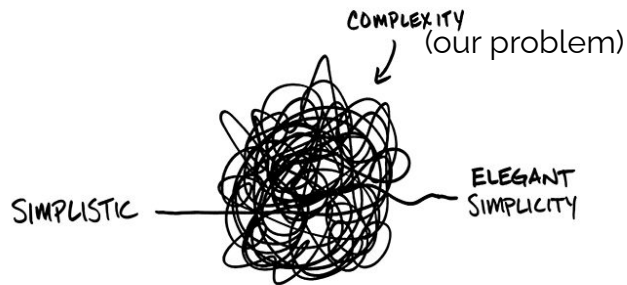
- Allocation: artifacts assigned to each robot.
- Previous approach:** pre-allocated tasks to robots prior to training.
- Novel approach:** robots must learn to allocate tasks to themselves and complete; applies to search phase.
 - Proximity Rewarding
 - Punish No Allocation
 - Reset upon failure during training
 - Incentivise Cooperation

time	Actions	Rewards
1	['right', 'right']	[-1.0, -1.0]
2	['up', 'up']	[-1.0, -1.0]
3	['allocate', 'up']	[50.0, -1.0]
4	['up', 'allocate']	[0.0, 50.0]



Removing Task Allocation Prior to Training

To illustrate the complexity of the problem, we now show cases in which agents did not learn to complete the tasks.



*complex neural network and correct reward mechanism

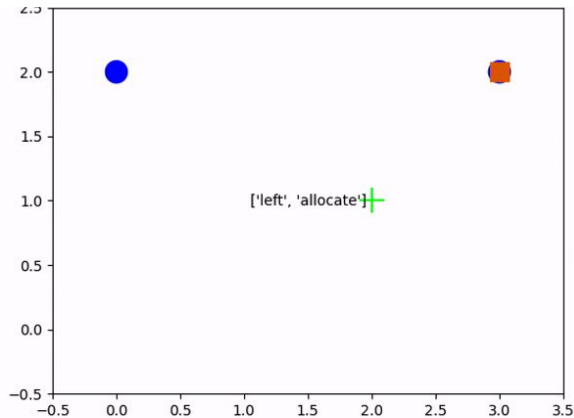
Punish No Allocation (Search Phase)

Control: fully observable, only agent's initial position random, 1000 simulations, 4x3 grid size, 2 agents, 1 object

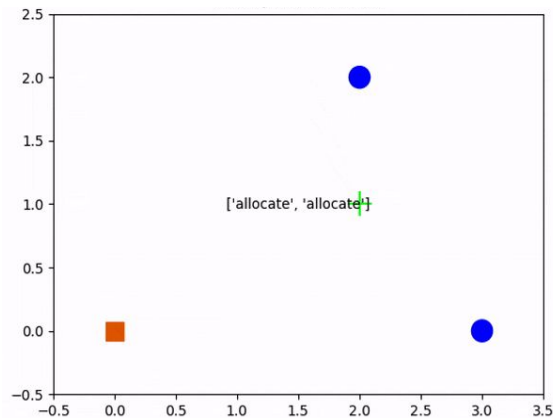
Training after reset
increases probability
agent stays with
artifact

Training Time	rewards	successes	Tasks not completed
1,000,000	-38.4/0.0/8.0%	80	920
10,000,000	-41.1/0.0/8.4%	84	916
20,000,000	-89.0/0.0/7.5%	75	925

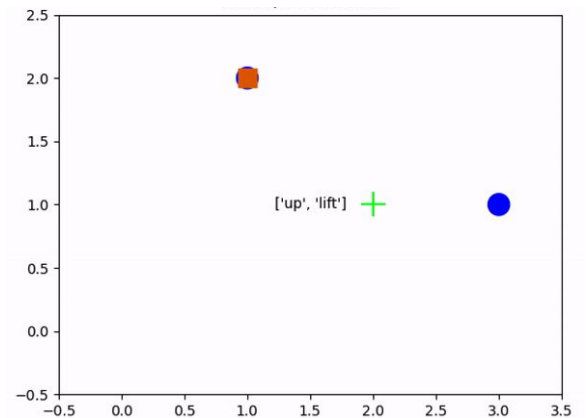
Training time: 1 million



Training time: 10 million

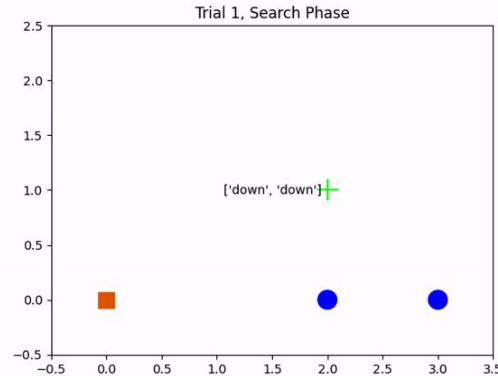
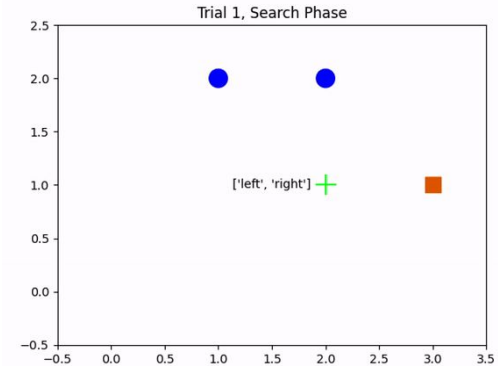


Training time: 20 million



No Pre Allocation of Tasks Prior to Training (Search Phase)

- **Experiment 1:** Episode is **reset** if agent moves away from an artifact after finding it (without assigning).
 - Forces agent to continue training
- **Experiment 2:** Agent is **partially rewarded** for going to an object and **largely rewarded** for assigning itself to an object
 - Result: Agents move back and forth to and from the object.
 - Fail to complete task.
- Agents learn suboptimal path due to **local minima**.
- Future Tests
 - **More robots and more artifacts:** if all robots allocate themselves to different artifacts, no cooperation occurs.
 - Engineer reward function



Major Challenges

- Training time: 10,000,000 time steps (on average) for loss function to converge (5-10 hours)
 - Detect and stop training once convergence is achieved
- Difficult to explain results and track down causation
 - Failure could be due to training time, or architecture of current system.
- Seemingly simple but complex problem to solve
- Engineering reward mechanism
 - Drives learning
 - Ex: Robots learn to go back and forth from and to and object if they are rewarded for finding an object.



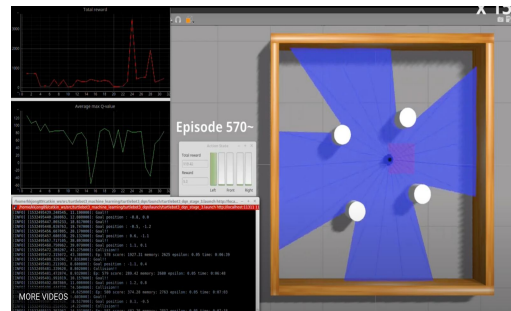
Condition	Reward
Artifact found and reached (search phase complete)	50
Artifact moved to goal (haul phase complete)	100
Collision with wall	-5
Action resulting in none of the above	-1

Ongoing Work: Physical Robot Simulations

Physical robot simulations to replicate existing system and to ensure practical applications and usage of the system.

Currently, the empirical evidence is limited to terminal and low-fidelity graphical output.

- ▣ Webots
 - High-fidelity physical robot simulator
 - Deepbots
- ▣ Gazebo
 - Physical robot simulator
 - Extendable to real robots through ROS



Conclusions & Future Work

- ▣ My work removes constraints and increases generalizability of the program: multiple agents, objects, no pre-allocation dependency.
- ▣ My work showcases the need to combine the neural networks.
 - Failure cases illustrate complexity of the problem.
 - Currently, [separate neural networks are used for each task](#).
 - Issue: dependency on pre-training task allocation.
 - Ideas: use complex network
 - Adapt Transformers with MAS; promising due to popularity with NLP.
 - Attention Mechanisms
 - LSTM does not achieve convergence in experiments we have run.
- ▣ Compare performance against other algorithms, such as Proximal Policy Optimization (PPO).

1 Neural Network – Results IP

	2 agents, 1 artifact	1 agent, 1 artifact
Total time steps	1,000,000	1,000,000
Number of artifact visits by any robot	77,126	362,298
Number of times any artifact is moved to goal	9	17,106
Number of times all artifacts moved to goal	9	17,106
Last time step an artifact is moved to the goal	894,221	999,617
Action - Count	Up - 4,456 Down - 7,274 Left - 6,999 Right - 4,692 Lift - 11,183 Stay - 6,506	Up - 29,897 Down - 108,487 Left - 48,870 Right - 99,591 Lift - 29,709 Stay - 30,029
First artifact visit time step	23	0

Reward Mechanism:
reward for lifting a
box, timeout when
robot stays at a box

More layers/nodes in
neural network



“

*“Research” means “to search again.” Why not?
Sometimes, a new interpretation emerges that is of
vast importance.*

- Isaac Asimov

Acknowledgements

Nabil Khan

khan0096@umn.edu

Professor Gini

gini@umn.edu

Angel Sylvester

sylve057@umn.edu

Parents

Chakradhari Ratala and
Sailaja Ratala

Ebasa Temesgen

temes021@umn.edu

***Swarm Lab
group members***

swarm@umn.edu

***Caltech sfp
office***

sfp@caltech.edu

Questions?

Thank you

No pre-allocation: Search phase

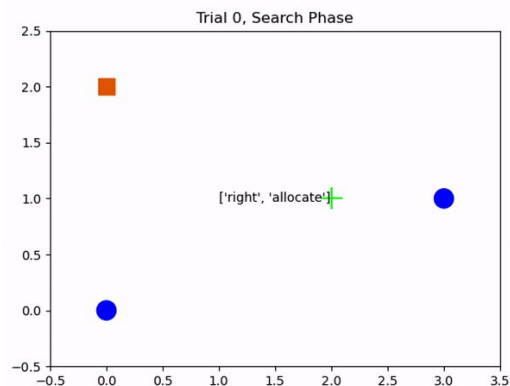
Control: fully observable, only agent's initial position random, 1000 simulations, 4x3 grid size

Agents	Objects	Training Time	rewards	successes	failures
1	1	100,000	-23.1/0.0/7.5%	75	925
2	1	10,000,000	-20.8/0.0/7.3%	73	927
2	1	20,000,000	-60.7/0.0/8.7%	87	913

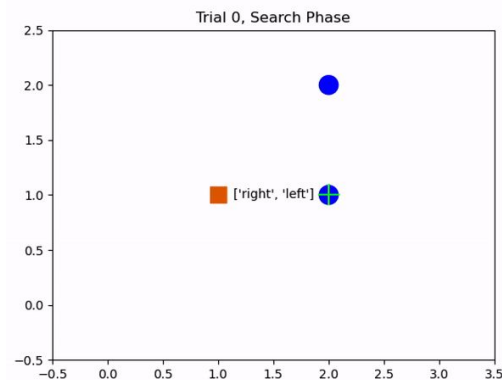
2 agents 1 artifact

Training time

10,000,000



20,000,000



No Pre-Allocation, Reset Episode Upon Failure during Training

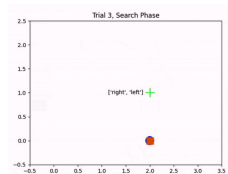
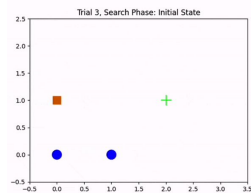
If agent moves away from an artifact after finding it, without assigning itself to it, we reset the episode, forcing agent to continue training.

- Agent perform less than optimal actions; only step back and forth or keeps calling assign.
- Stuck at a local maxima.

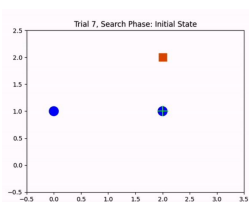
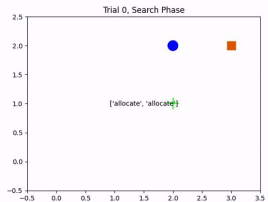
Training time: 10,000,000

Training time: 20,000,000

Pos Mode 1: Agent
Random



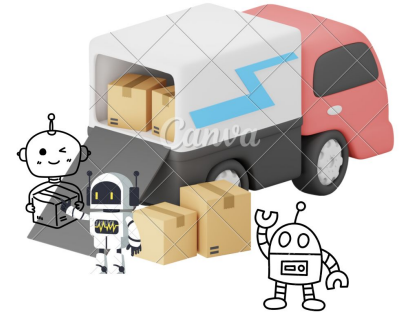
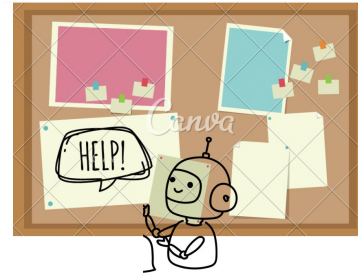
Pos Mode 2: Agent
Random



Position Mode	Training Time	Search Reward	Tasks Completed (%)	Complete Successes	Complete Failures
1	10,000,000	-44.3	8.2	82	918
1	20,000,000	-38	8.4	84	916
2	10,000,000	-45	8	80	920
2	20,000,000	-38.1	9.7	97	903

Communication

- With Communication
 - Environment changes over time, updating mechanism required
 - Learned Communication
 - Pre-existing Communication
 - Like posting on a bulletin board
- Without Communication
 - Updating Reward
 - Removing pre-allocation



Proximity Reward

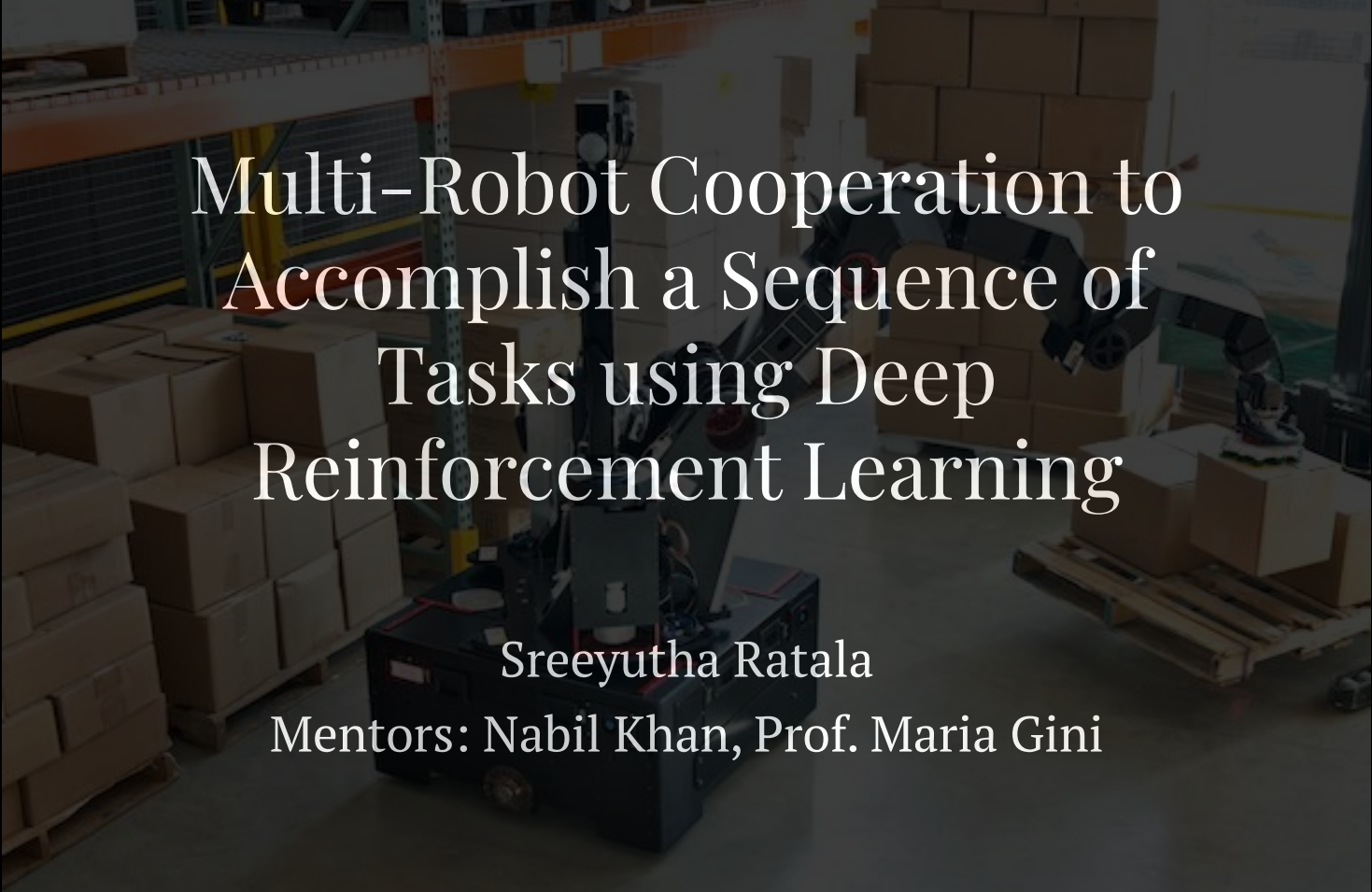
Robots go back and forth between artifact to gain reward for going closer to an artifact.

*Caltech SURF 2023,
off-campus at the University of Minnesota*

Sreeyutha Ratala

sratala@caltech.edu

Sophomore at Caltech, CS

The background image shows a warehouse environment. In the foreground, a black robotic forklift is positioned, carrying a stack of cardboard boxes. To its right, another robotic forklift is visible, also handling boxes. The warehouse is filled with numerous stacks of cardboard boxes on pallets, and high industrial shelving units are visible in the background. The entire scene is dimly lit, with the text overlaid in a bright, white font.

Multi-Robot Cooperation to Accomplish a Sequence of Tasks using Deep Reinforcement Learning

Sreeyutha Ratala

Mentors: Nabil Khan, Prof. Maria Gini

Contents

EDIT IN GOOGLE SLIDES

Click on the button under the presentation preview that says "Use as Google Slides Theme".

You will get a copy of this document on your Google Drive and will be able to edit, add or delete slides.

You have to be signed in to your Google account.

EDIT IN POWERPOINT®

Click on the button under the presentation preview that says "Download as PowerPoint template". You will get a .pptx file that you can edit in PowerPoint.

Remember to download and install the fonts used in this presentation (you'll find the links to the font files needed in the [Presentation design slide](#))

More info on how to use this template at
www.slidescarnival.com/help-use-presentation-template

This template is free to use under [Creative Commons Attribution license](#). You can keep the Credits slide or mention SlidesCarnival and other resources used in a slide footer.